GUIDE DE REVISION

LINUX

Systeme d'Exploitation - Cours Complet

Chapitres couverts :

Ch.1 Gestion des fichiers et repertoires - Ch.2 Redirections, tubes et filtres
Ch.3 Gestion des utilisateurs et groupes - Ch.4 Permissions et ACL
Ch.5 Gestion des processus - Ch.6 Programmation Shell

Contenu :
Cours resume - Definitions - Commandes essentielles - Exercices corriges - QCM - Glossaire

Document de revision - Examens de fin de semestre

Base sur les cours, TD et notes de revision officiels

■ TABLE DES MATIERES

1. RESUME DU COURS 3

2. CHAPITRE 1 - Gestion des Fichiers et Repertoires 4

2.1 Structure du systeme de fichiers Linux 4

2.2 Commandes essentielles 4

2.3 Commande find 5

3. CHAPITRE 2 - Redirections, Tubes et Filtres 6

3.1 Redirections d'entree/sortie 6

3.2 Tubes (pipes) 6

3.3 Filtres : grep, sed, cut, sort, uniq, tr 7

4. CHAPITRE 3 - Gestion des Utilisateurs et Groupes 9

4.1 Fichiers de configuration 9

4.2 Commandes utilisateurs 9

4.3 Commandes groupes 10

4.4 Gestion des mots de passe (passwd, chage) 10

5. CHAPITRE 4 - Permissions et ACL 11

5.1 Permissions standard (chmod, chown, chgrp) 11

5.2 Permissions speciales (SUID, SGID, Sticky) 12

5.3 Listes de Controle d'Acces (ACL) 13

6. CHAPITRE 5 - Gestion des Processus 14

6.1 Concepts de base 14

6.2 Commandes de gestion des processus 14

6.3 Signaux et gestion des jobs 15

7. CHAPITRE 6 - Programmation Shell (Bash) 16

7.1 Variables et parametres 16

7.2 Structures de controle 16

7.3 Fonctions et tableaux 17

8. EXERCICES CORRIGES (TD de Revision) 18

9. QUESTIONS D'EXAMEN FREQUENTES 22

10. FICHE DE REVISION RAPIDE 23

11. GLOSSAIRE LINUX 25

■ 1. RESUME DU COURS

Ce guide couvre l'integralite du module **Systeme d'Exploitation Linux**. Linux est un systeme d'exploitation libre base sur le noyau Linux (Linus Torvalds, 1991). Il est multi-utilisateur, multi-tache, et repose sur une philosophie : *"Tout est fichier"*.

◆ Architecture generale de Linux

Couche	Role
Materiel (Hardware)	CPU, RAM, disques, peripheriques
Noyau (Kernel)	Gestion memoire, processus, drivers, appels systeme
Shell	Interface entre l'utilisateur et le noyau (bash, sh, zsh)
Applications	Programmes, commandes, services

◆ Les 6 themes du cours

[F] **Fichiers & Repertoires**
Navigation, creation, copie, deplacement, recherche avec find

[R] **Redirections & Filtres**
stdin/stdout/stderr, pipes |, grep, sed, cut, sort, uniq, tr

[U] **Utilisateurs & Groupes**
useradd/mod/del, groupadd/mod/del, /etc/passwd, /etc/shadow

[P] **Permissions & ACL**
chmod, chown, chgrp, SUID/SGID/Sticky, setfacl/getfacl

[Pr] **Processus**
ps, top, kill, jobs, bg, fg, nice, nohup

[S] **Shell Bash**
Variables, if/for/while, fonctions, scripts .sh

[+] A RETENIR

Linux = Noyau + Shell + Applications. Le shell est l'interpreteur de commandes.

Tout en Linux est fichier : peripheriques, sockets, repertoires...

Le superutilisateur est root (UID=0). Il a tous les droits.

CHAPITRE 1

Gestion des Fichiers et Repertoires

◆ 1.1 Structure du systeme de fichiers Linux (FHS)

L'arborescence Linux est unique, enracinee en **/** (racine). Voici les repertoires principaux :

Repertoire	Role
`/`	Racine de toute l'arborescence
`/bin`	Commandes essentielles (ls, cp, mv, rm...)
`/sbin`	Commandes systeme (mount, fsck, ifconfig...)
`/etc`	Fichiers de configuration systeme
`/home`	Repertoires personnels des utilisateurs
`/root`	Repertoire de l'utilisateur root
`/tmp`	Fichiers temporaires (effaces au redemarrage)
`/usr`	Programmes et donnees partagees
`/var`	Donnees variables (logs, spool, mail...)
`/dev`	Fichiers speciaux (peripheriques)
`/proc`	Systeme de fichiers virtuel (infos noyau/processus)
`/lib`	Bibliotheques partagees
`/mnt`	/media : Points de montage

◆ 1.2 Commandes essentielles de navigation et gestion

■ Navigation

Commande	Description
`pwd`	Affiche le repertoire courant (Print Working Directory)
`cd /chemin`	Change de repertoire (absolu)
`cd ..`	Remonte d'un niveau
`cd ~`	Va dans le repertoire personnel
`cd -`	Revient au repertoire precedent
`ls`	Liste le contenu du repertoire courant

Commande	Description
`ls -l`	Affichage long (permissions, proprietaire, taille, date)
`ls -la`	Affiche aussi les fichiers caches (commencant par .)
`ls -lh`	Taille en format lisible (Ko, Mo...)
`ls -lR`	Recursif (liste les sous-repertoires)
`tree`	Affiche l'arborescence en arbre

■ Creation et manipulation

Commande	Description
`mkdir rep`	Cree un repertoire
`mkdir -p a/b/c`	Cree les repertoires intermediaires si necessaire
`touch fichier`	Cree un fichier vide ou met a jour la date
`cp src dst`	Copie un fichier
`cp -r src/ dst/`	Copie recursive (repertoires)
`mv src dst`	Deplace ou renomme
`rm fichier`	Supprime un fichier
`rm -r rep/`	Supprime un repertoire et son contenu
`rm -rf rep/`	Force la suppression sans confirmation
`rmdir rep`	Supprime un repertoire vide uniquement
`ln -s cible lien`	Cree un lien symbolique
`ln cible lien`	Cree un lien physique (hard link)

■ Consultation de fichiers

Commande	Description
`cat fichier`	Affiche le contenu complet
`more fichier`	Affiche page par page (navigation avec Espace)
`less fichier`	Affiche avec navigation avant/arriere (q pour quitter)
`head -n N fich`	Affiche les N premieres lignes (defaut: 10)
`tail -n N fich`	Affiche les N dernieres lignes
`tail -f fich`	Suit les ajouts en temps reel (logs)

Commande	Description
`wc fich`	Compte lignes (-l), mots (-w), caracteres (-c)
`file fich`	Determine le type de fichier
`stat fich`	Informations detaillees (taille, inode, dates...)

◆ 1.3 Commande find - Recherche avancee

La commande **find** est incontournable pour les examens. Syntaxe : `find [chemin] [options] [action]`

Exemple	Description
`find / -name "*.c"`	Fichiers dont le nom finit par .c (toute l'arborescence)
`find /usr -name "*.c" 2>/dev/null`	Idem, erreurs redirigees vers /dev/null
`find /home -name "[a-z,P]*"`	Nom commencant par minuscule ou P majuscule
`find /usr -size +1M`	Taille > 1 Mo (+1M), < 1Mo (-1M), exactement (1M)
`find /usr -size +1M -a -perm 755`	Taille > 1Mo ET droits 755 (-a = AND)
`find / -user student`	Appartenant a l'utilisateur student
`find / -user student -a -perm 666`	Appartenant a student ET droits 666
`find / -user student -a -perm 666 | wc -l`	Compter ces fichiers
`find /home -type f`	Seulement les fichiers (-type f)
`find /home -type d`	Seulement les repertoires (-type d)
`find /home -type d -empty`	Repertoires vides
`find /home -type d -empty -exec rmdir {} \;`	Supprimer les repertoires vides
`find /usr -name "*[a-z]" -exec cp {} /dest \;`	Copier les fichiers dont le nom finit par minuscule
`find /usr -size +1M -a -perm 755 > names`	Sauvegarder le resultat dans un fichier
`find / -mtime -7`	Modifies dans les 7 derniers jours
`find / -newer ref.txt`	Plus recents que ref.txt

[+] A RETENIR

find chemin : cherche dans le chemin donne et tous ses sous-repertoires.

-a signifie AND (ET logique), -o signifie OR (OU logique).

-exec commande {} \; : execute une commande sur chaque resultat trouve.

2>/dev/null : redirige les messages d'erreur vers la poubelle (evite "Permission denied").

Pour compter les resultats, utiliser | wc -l a la fin.

[!] ATTENTION AUX PIEGES

find / -name "*.c" cherche depuis la racine = lent ! Preferer find /chemin/precis.

Les guillemets autour des motifs sont necessaires : -name "*.c" et non -name *.c

Le \; final dans -exec est obligatoire (point-virgule echappe).

-perm 755 cherche exactement 755. -perm -755 cherche au moins ces droits.

CHAPITRE 2

Redirections, Tubes et Filtres

◆ 2.1 Redirections d'entree/sortie

Chaque processus Linux dispose de 3 flux standard :

Descripteur	Nom	Defaut
0	**stdin**	Entree standard (clavier par defaut)
1	**stdout**	Sortie standard (ecran par defaut)
2	**stderr**	Sortie d'erreur (ecran par defaut)

Syntaxe	Description
`cmd > fichier`	Redirige stdout vers fichier (ecrase)
`cmd >> fichier`	Redirige stdout vers fichier (ajoute a la fin)
`cmd < fichier`	Redirige stdin depuis fichier
`cmd 2> fichier`	Redirige stderr vers fichier
`cmd 2>> fichier`	Redirige stderr vers fichier (ajoute)
`cmd > fich 2>&1`	Redirige stdout ET stderr vers fichier
`cmd 2>/dev/null`	Supprime les erreurs (/dev/null = poubelle)
`cmd 1>/dev/null 2>&1`	Supprime toute la sortie

[+] A RETENIR

Sortie 1 = sortie standard (stdout). Sortie 2 = sortie d'erreur (stderr).

> ecrase le fichier. >> ajoute au fichier. < lit depuis un fichier.

2>/dev/null : tres souvent utilise dans les exercices d'examen avec find.

◆ 2.2 Tubes (pipes) |

Le pipe **|** connecte la sortie d'une commande a l'entree de la suivante :

```
# Compter les lignes d'un fichier
cat fichier | wc -l

# Trier et supprimer les doublons
cat fichier | sort | uniq

# Afficher la 8eme ligne d'un fichier
cat fichier | head -n 8 | tail -1
```

```
# Extraire les GIDs uniques de /etc/passwd
cut -d: -f4 /etc/passwd | sort -n | uniq

# Rechercher les processus d'un utilisateur
ps aux | grep username

# Compter des fichiers correspondant a une recherche
find / -user student -a -perm 666 | wc -l
```

◆ 2.3 Filtres - grep, sed, cut, sort, uniq, tr

■ grep - Recherche de motifs dans des fichiers

Commande	Description
`grep motif fich`	Lignes contenant le motif
`grep -i motif fich`	Insensible a la casse
`grep -v motif fich`	Lignes NE contenant PAS le motif
`grep -n motif fich`	Affiche le numero de ligne
`grep -c motif fich`	Compte le nombre de lignes correspondantes
`grep -r motif /rep`	Recherche recursive dans un repertoire
`grep "^motif" fich`	Lignes COMMENCANT par motif
`grep "motif$" fich`	Lignes FINISSANT par motif
`grep "e$" couleurs`	Lignes finissant par e (exemple du TD)

■ sed - Editeur de flux (Stream EDitor)

sed traite les fichiers ligne par ligne. Tres utilise en examen !

Commande sed	Description
`sed 's/ancien/nouveau/g' fich`	Remplace toutes les occurrences
`sed 's/ancien/nouveau/' fich`	Remplace seulement la premiere occurrence par ligne
`sed '/motif/d' fich`	Supprime les lignes contenant le motif
`sed '/^#/d' fich`	Supprime les lignes commencant par # (commentaires)
`sed -n '/motif/p' fich`	Affiche uniquement les lignes correspondant au motif
`sed -n '/e$/p' fich`	Affiche les lignes finissant par e

Commande sed	Description
`sed 's/#/ /g' fich`	Remplace # par un espace (exemple TD)
`sed 's/?/*/g' fich`	Remplace ? par *
`sed -i 's/.*/\U&/g' fich`	Convertit tout en majuscules en place (-i)
`sed '2,5d' fich`	Supprime les lignes 2 a 5
`sed '1i\nouvelle ligne' fich`	Insere une ligne avant la ligne 1

■ cut - Extraction de colonnes

Commande	Description
`cut -d: -f1 /etc/passwd`	Extraire le champ 1 (login) avec delimiteur :
`cut -d: -f4 /etc/passwd`	Extraire le GID (champ 4)
`cut -d: -f1,3 /etc/passwd`	Extraire les champs 1 et 3
`cut -c1-10 fichier`	Extraire les caracteres 1 a 10 de chaque ligne
`cut -d: -f4 /etc/passwd | sort -n | uniq`	GIDs uniques tries numeriquement

■ sort - Tri

Commande	Description
`sort fichier`	Tri alphabetique croissant
`sort -r fichier`	Tri inverse
`sort -n fichier`	Tri numerique
`sort -u fichier`	Tri et suppression des doublons
`sort -k2 fichier`	Tri selon la 2eme colonne
`sort -t: -k3 -n fich`	Tri numerique sur la 3eme colonne (delimiteur :)

■ uniq - Suppression des doublons

Commande	Description
`uniq fichier`	Supprime les lignes CONSECUTIVES dupliquees
`sort fich | uniq`	Supprime TOUS les doublons (tri prealable necessaire)
`sort fich | uniq -c`	Compte les occurrences de chaque ligne

Commande	Description
`sort fich | uniq -d`	Affiche uniquement les lignes dupliquees

■ tr - Transformation de caracteres

Commande	Description
`tr "a-z" "A-Z" < fich`	Convertit minuscules en majuscules
`tr "A-Z" "a-z" < fich`	Convertit majuscules en minuscules
`tr -d "\n" < fich`	Supprime les sauts de ligne
`tr ":" " " < fich`	Remplace : par espace
`tr "[a-z]" "[A-Z]" fich > fich2`	Sortie vers un nouveau fichier

[+] A RETENIR

grep = chercher dans un fichier. sort = trier. cut = extraire des colonnes.

find = chercher des fichiers dans l'arborescence. uniq = enlever les doublons.

sed 's/a/b/g' remplace a par b partout (g = global). Sans g = premiere occurrence.

Pipeline typique examen : cat fich | sort | uniq > new.fich 2>/dev/null

[!] ATTENTION AUX PIEGES

uniq seul ne supprime que les doublons CONSECUTIFS. Toujours faire sort | uniq.

tr lit depuis stdin : tr "a-z" "A-Z" < fichier (pas directement le nom du fichier).

sed -i modifie le fichier en place. Attention, cette modification est irreversible !

CHAPITRE 3

Gestion des Utilisateurs et Groupes

◆ 3.1 Fichiers de configuration

■ /etc/passwd - Base de donnees des utilisateurs

Format : **login:x:uid:gid:commentaire:home_directory:shell**

```
# Exemple de ligne /etc/passwd
john:x:1001:1001:John Doe:/home/john:/bin/bash
# | | | | | | |
# | | | | | | +-- Shell de connexion
# | | | | | +------------- Repertoire home
# | | | | +------------------------ Commentaire (GECOS)
# | | | +------------------------------ GID du groupe primaire
# | | +-------------------------------- UID (User ID)
# | +------------------------------------ x = mot de passe dans /etc/shadow
# +---------------------------------------- Nom de connexion (login)
```

■ /etc/shadow - Mots de passe chiffres

Format : **login:mdp_chiffre:derniere_modif:min:max:warn:inactif:expir:reserve**

```
# Champs de /etc/shadow
john:$6$hash...:19000:0:90:7:30:20000:
# | | | | | | |
# | | | | | | +- Date expiration compte
# | | | | | +----- Jours inactivite apres expiration
# | | | | +------- Jours d'avertissement avant expiration
# | | | +---------- Max jours validite mot de passe (-M)
# | | +------------ Min jours entre changements (-m)
# | +-------------- Jours depuis 01/01/1970 (dernier changement)
```

■ /etc/group - Base de donnees des groupes

Format : **nom_groupe:x:gid:membres**

```
developers:x:1002:john,alice,bob
# | | | |
# | | | +-- Liste des membres (separes par virgule)
# | | +--------- GID (Group ID)
# | +------------ x = mot de passe groupe (rarement utilise)
# +--------------------- Nom du groupe
```

■ Fichiers de configuration complementaires

Fichier	Role
`/etc/login.defs`	Parametres : UID_MIN, UID_MAX, GID_MIN, GID_MAX, PASS_MAX_DAYS...
`/etc/skel`	Repertoire modele : contenu copie dans le home a la creation (.bashrc, .bash_profile)

Fichier	Role
`/etc/default/useradd`	Valeurs par defaut : SHELL, BASE (/home), EXPIRE...
`/etc/sudoers`	Droits sudo (editer avec visudo)

◆ 3.2 Commandes de gestion des utilisateurs

Commande	Description
`useradd nom`	Cree un utilisateur avec les valeurs par defaut
`useradd -l login nom`	Specifie le nom de connexion
`useradd -p mdp nom`	Definit le mot de passe (chiffre)
`useradd -u uid nom`	Specifie l'UID
`useradd -g grp nom`	Definit le groupe PRIMAIRE
`useradd -G g1,g2 nom`	Definit les groupes SECONDAIRES
`useradd -c "Com" nom`	Ajoute un commentaire (GECOS)
`useradd -d /home/x nom`	Specifie le repertoire home
`useradd -s /bin/sh nom`	Definit le shell de connexion
`useradd -m nom`	Cree le repertoire home (copie /etc/skel)
`useradd engineer -G techdocs -s /bin/sh -g consultants`	Exemple complet : engineer avec groupe primaire consultants, secondaire techdocs, shell /bin/sh
`usermod -aG wheel engineer`	Ajoute engineer au groupe wheel (sudo) - SANS -a, remplace les groupes!
`usermod -l new old`	Change le nom de connexion
`usermod -L user`	Verrouille le compte (Lock)
`usermod -U user`	Deverrouille le compte (Unlock)
`userdel user`	Supprime l'utilisateur (conserve le home)
`userdel -r user`	Supprime l'utilisateur ET son repertoire home
`su - user`	Change d'utilisateur avec son environnement complet
`id user`	Affiche UID, GID et groupes de l'utilisateur
`whoami`	Affiche l'utilisateur courant
`who / w`	Affiche les utilisateurs connectes

◆ 3.3 Commandes de gestion des groupes

Commande	Description
`groupadd groupe`	Cree un groupe
`groupadd -g gid groupe`	Cree un groupe avec un GID specifique
`groupmod -n newname old`	Renomme un groupe (-n)
`groupmod -g newgid grp`	Change le GID (-g)
`groupdel groupe`	Supprime un groupe
`gpasswd -a user grp`	Ajoute un utilisateur a un groupe
`gpasswd -d user grp`	Retire un utilisateur d'un groupe
`newgrp groupe`	Change le groupe actif pour la session courante
`groups user`	Affiche les groupes d'un utilisateur

◆ 3.4 Gestion des mots de passe - passwd et chage

Commande	Description
`passwd user`	Definit/change le mot de passe de user (root)
`passwd`	Change son propre mot de passe
`passwd -l user`	Verrouille le compte
`passwd -u user`	Deverrouille le compte
`passwd -e user`	Force le changement au prochain login

chage - Gestion de la politique d'expiration des mots de passe :

Commande	Description
`chage -E 2024-12-12 user`	Date d'expiration du COMPTE (format YYYY-MM-DD)
`chage -M 90 user`	Nombre MAX de jours de validite du mot de passe
`chage -m 7 user`	Nombre MIN de jours entre deux changements
`chage -W 7 user`	Jours d'avertissement avant expiration
`chage -d 0 user`	Force le changement de MDP a la prochaine connexion
`chage -l user`	Affiche les informations d'expiration
`chage -I 30 user`	Jours d'inactivite apres expiration avant desactivation

[+] A RETENIR

Structure /etc/passwd : login:x:uid:gid:commentaire:home:shell

Structure /etc/group : nom:x:gid:membres

useradd -g = groupe primaire, -G = groupes secondaires (virgule separes).

usermod -aG : le -a est CRUCIAL - sans lui, les anciens groupes sont supprimes.

userdel -r supprime le home. Sans -r, le home reste.

chage -d 0 force le changement de MDP a la premiere connexion.

chage -E = expiration compte. chage -M = max jours mot de passe.

[!] ATTENTION AUX PIEGES

usermod -G (sans -a) REMPLACE tous les groupes secondaires ! Toujours utiliser -aG.

groupdel echoue si le groupe est le groupe primaire d'un utilisateur.

Ne jamais editer /etc/passwd directement : utiliser vipw. Pour sudoers : visudo.

CHAPITRE 4

Permissions et Listes de Controle d'Acces (ACL)

◆ 4.1 Permissions standard Linux

■ Lecture d'une ligne ls -l

Exemple : `-rwxr--r-x . user1 user1 fich`

```
- rwx r-- r-x . user1 user1 fich
| | | | | | | |
| | | | | | | +-- Nom du fichier
| | | | | | +--------- Groupe proprietaire
| | | | | +---------------- Proprietaire (owner)
| | | | +---------------------- . = contexte SELinux ou ACL
| | | +------------------------ Droits des AUTRES (others)
| | +------------------------------ Droits du GROUPE (group)
| +-------------------------------------- Droits du PROPRIETAIRE (user)
+---------------------------------------- Type : - fichier, d repert., l lien...

=> En notation octale : rwx=7, r--=4, r-x=5 => 745
```

■ Valeurs des droits

Droit	Lettre	Valeur octale	Sur fichier	Sur repertoire
Read	**r**	**4**	Lire le contenu	Lister (ls)
Write	**w**	**2**	Modifier/ecrire	Creer/supprimer des fichiers
Execute	**x**	**1**	Executer le fichier	Acceder (cd) et traverser
Aucun	**-**	**0**	Aucun droit	Aucun droit

■ chmod - Modification des permissions

Mode symbolique : chmod [u|g|o|a][+|-|=][r|w|x] fichier

Commande	Description
`chmod u+x fich`	Ajoute le droit d'execution au proprietaire
`chmod g-w fich`	Retire le droit d'ecriture au groupe
`chmod o=x fich`	Fixe les droits des autres a --x seulement
`chmod u=rwx fich`	Fixe les droits du proprietaire a rwx
`chmod u-wx fich`	Retire w et x au proprietaire (equivalent a chmod u=r)

Commande	Description
`chmod u=r fich`	Proprietaire = r-- (lecture seulement)
`chmod g-r fich`	Retire r au groupe -> ---
`chmod a+r fich`	Ajoute r pour tous (all)
`chmod o+t /rep`	Active le sticky bit sur un repertoire
`chmod g+s /rep`	Active le SGID sur un repertoire
`chmod u+s prog`	Active le SUID sur un programme
`chmod -R 755 /rep`	Applique recursivement

Mode octal : chmod NNN fichier (N = somme r+w+x par categorie)

Commande	Resultat / Description
`chmod 755 fich`	rwxr-xr-x : proprietaire=rwx(7), groupe=r-x(5), autres=r-x(5)
`chmod 644 fich`	rw-r--r-- : proprietaire=rw-(6), groupe=r--(4), autres=r--(4)
`chmod 600 fich`	rw------- : proprietaire seul peut lire/ecrire
`chmod 777 fich`	rwxrwxrwx : tous les droits pour tous (DANGEREUX !)
`chmod 000 fich`	--------- : aucun droit
`chmod 700 fich`	rwx------ : proprietaire seul, tous droits
`chmod 745 fich`	rwxr--r-x : exemple du cours (octal de -rwxr--r-x)
`chmod 666 fich`	rw-rw-rw- : lecture/ecriture pour tous

■ chown et chgrp - Changement de proprietaire

Commande	Description
`chown user fich`	Change le proprietaire du fichier
`chown user:grp fich`	Change proprietaire ET groupe
`chown :grp fich`	Change uniquement le groupe (= chgrp)
`chown -R user /rep`	Recursif sur tout le repertoire
`chgrp grp fich`	Change le groupe proprietaire
`chgrp -R grp /rep`	Recursif : change le groupe de rep et tout son contenu
`chgrp -R user2 rep`	Exemple du cours : changer le groupe de rep et son contenu

◆ 4.2 Permissions speciales : SUID, SGID, Sticky Bit

Bit	chmod	Octal	Sur fichier executable	Sur repertoire
SUID	u+s	4xxx	S'execute avec les droits du PROPRIETAIRE	(effet non standard)
SGID	g+s	2xxx	S'execute avec les droits du GROUPE	Les nouveaux fichiers heritent du GROUPE du repertoire
Sticky	o+t	1xxx	Obsolete pour les fichiers	Seul le proprietaire peut supprimer ses fichiers

```
# Reconnaitre les permissions speciales dans ls -l
drwxrwsr-t operator1 database1 rep
# | |
# | +-- t (sticky) : protection suppression
# +------ s (SGID) : les nouveaux fichiers heritent du groupe "database1"

-rwSr--rw- consultant1 database1 file5
# |
# +-- S majuscule = SUID actif MAIS x absent (execution impossible !)
# s minuscule = SUID actif ET x present
```

[+] A RETENIR

SGID sur repertoire (chmod g+s /rep) : tous les fichiers crees heritent du groupe du repertoire.

Sticky bit (chmod o+t /rep) : dans /tmp et repertoires partages, seul le proprietaire supprime ses fichiers.

S majuscule = bit special actif MAIS droit x absent. s minuscule = bit special ET x actif.

drwxrwsr-t en octal = 3775 (sticky=1, SGID=2 => 3xxx, puis rwx=7, rws=7, rwt=5).

[!] ATTENTION AUX PIEGES

Un S majuscule (SUID/SGID) signifie que le bit est positionne mais x est absent -> inutile en pratique.

Le Sticky bit sur un repertoire NE protege PAS contre la modification du contenu des fichiers, seulement contre leur SUPPRESSION.

Pour supprimer un fichier dans un repertoire, il faut le droit w+x sur le REPERTOIRE parent, pas sur le fichier lui-meme.

◆ 4.3 Listes de Controle d'Acces (ACL)

Les ACL permettent d'attribuer des droits a des utilisateurs ou groupes **supplementaires**, au-dela du modele UGO classique.

■ getfacl et setfacl

Commande	Description
`getfacl fich`	Affiche l'ACL complete d'un fichier/repertoire

Commande	Description
`setfacl -m u:user:rwx fich`	Attribue rwx a l'utilisateur user
`setfacl -m u:user2:rwx /home/techdocs`	Exemple du TD : rwx pour user2 sur /home/techdocs
`setfacl -m g:tekup:rw- fich`	Attribue rw- au groupe tekup
`setfacl -m g:tic:rwx fich`	Attribue rwx au groupe tic
`setfacl -m o::r-- fich`	Definit les droits des autres
`setfacl -x u:user fich`	Supprime l'ACL de l'utilisateur user
`setfacl -b fich`	Supprime TOUTES les ACL du fichier
`setfacl -R -m g:grp:rwx /rep`	Applique recursivement a tout le repertoire
`setfacl -d -m g:grp:rwx /rep`	Definit une ACL par DEFAUT (pour nouveaux fichiers)

[i] RAPPEL

Quand un fichier a des ACL, ls -l affiche un + apres les permissions : -rw-rw-r--+

La priorite est : entree ACL utilisateur > masque ACL > permissions de base.

setfacl et chown/chmod/chgrp fonctionnent tous avec -R pour la recursivite.

CHAPITRE 5

Gestion des Processus

◆ 5.1 Concepts fondamentaux

Processus : instance d'un programme en cours d'execution. Chaque processus possede un **PID** (Process ID) unique, un **PPID** (Parent PID), et s'execute avec les droits d'un utilisateur.

■ Etats d'un processus

Etat	Description
`R (Running)`	En cours d'execution ou pret a s'executer
`S (Sleeping)`	En attente d'un evenement (sommeil interruptible)
`D (Disk sleep)`	Attente E/S disque (sommeil non interruptible)
`Z (Zombie)`	Termine mais non recupere par le pere
`T (Stopped)`	Arrete (signal SIGSTOP ou Ctrl+Z)
`I (Idle)`	Thread du noyau inactif

◆ 5.2 Commandes de surveillance et gestion

Commande	Description
`ps`	Processus du terminal courant
`ps aux`	Tous les processus (format BSD) : a=tous users, u=format detaille, x=sans terminal
`ps -ef`	Tous les processus (format UNIX standard)
`ps -ef | grep user`	Processus d'un utilisateur specifique
`top`	Affichage dynamique temps reel (q pour quitter)
`htop`	Version amelioree de top (interactive)
`pgrep nom`	Trouve le PID d'un processus par son nom
`pidof nom`	Affiche les PIDs d'un programme
`pstree`	Affiche l'arbre des processus

■ Signaux et kill

Commande	Description
`kill PID`	Envoie SIGTERM (15) au processus : demande d'arret propre

Commande	Description
`kill -9 PID`	Envoie SIGKILL (9) : arret force et immediat (ne peut etre ignore)
`kill -15 PID`	SIGTERM : arret gracieux (peut etre intercepte)
`kill -1 PID`	SIGHUP : rechargement de configuration (daemon)
`kill -2 PID`	SIGINT : equivalent Ctrl+C
`kill -19 PID`	SIGSTOP : pause du processus (equivalent Ctrl+Z)
`kill -18 PID`	SIGCONT : reprise d'un processus pause
`killall nom`	Envoie un signal a tous les processus de ce nom
`pkill motif`	Envoie un signal aux processus correspondant au motif

◆ 5.3 Gestion des taches (jobs) - avant/arriere-plan

Commande	Description
`commande &`	Lance en arriere-plan (background)
`Ctrl+Z`	Suspend le processus courant (met en pause)
`Ctrl+C`	Interrompt le processus courant (SIGINT)
`jobs`	Liste les jobs du terminal courant avec leur numero
`bg %N`	Relance le job N en arriere-plan
`fg %N`	Ramene le job N au premier plan
`fg`	Ramene le dernier job suspendu au premier plan
`nohup cmd &`	Lance en arriere-plan, immune au SIGHUP (survit a la deconnexion)
`wait`	Attend la fin de tous les processus fils

■ Priorite des processus

Commande	Description
`nice -n 10 cmd`	Lance cmd avec une priorite reduite (nice 10, moins prioritaire)
`nice -n -5 cmd`	Lance avec priorite elevee (necessite root pour valeur negative)
`renice 5 -p PID`	Change la priorite d'un processus existant
`renice -n 5 -u user`	Change la priorite de tous les processus d'un utilisateur

Valeurs de nice : de **-20** (plus haute priorite, reserve root) a **+19** (plus basse priorite). Valeur par defaut : 0.

[+] A RETENIR

PID = identifiant unique d'un processus. PPID = PID du processus pere.

kill -9 = SIGKILL = arret force (ne peut etre ignore par le processus).

Ctrl+Z = suspend (SIGSTOP). Ctrl+C = interrompt (SIGINT). & = arriere-plan.

nohup commande & : la commande continue meme apres la deconnexion de l'utilisateur.

jobs liste les taches du shell courant. fg/bg pour les gerer.

CHAPITRE 6

Programmation Shell (Bash)

◆ 6.1 Variables et parametres

■ Variables

```
#!/bin/bash
# Declaration et affectation (PAS d'espace autour de =)
nom="Alice"
age=25
compteur=0

# Utilisation ($ pour acceder a la valeur)
echo "Bonjour $nom"
echo "Vous avez ${age} ans" # {} pour delimiter

# Variables speciales
echo $0 # Nom du script
echo $1 # Premier argument
echo $2 # Deuxieme argument
echo $# # Nombre d'arguments
echo $@ # Tous les arguments (liste)
echo $* # Tous les arguments (chaine)
echo $$ # PID du script courant
echo $? # Code de retour de la derniere commande (0=succes)

# Substitution de commande
x=$(ls $2) # Modern syntax (recommande)
x=`ls $2` # Old syntax (backticks)

# Operations arithmetiques
((compteur=compteur+1)) # Arithmetic evaluation
((compteur++))
compteur=$(expr $compteur + 1) # Old style
let "compteur += 1"
```

◆ 6.2 Structures de controle

■ Condition if/elif/else

```
#!/bin/bash
if [ condition ]; then
# commandes si vrai
elif [ autre_condition ]; then
# commandes si autre condition vraie
else
# commandes sinon
fi

# Tests sur fichiers (test -options ou [ -options ])
if test -f "$2/$i"; then # test -f = fichier ordinaire
```

```
if [ -d "/home/rep" ]; then # -d = repertoire
if [ -e "fichier" ]; then # -e = existe
if [ -r "fichier" ]; then # -r = lisible
if [ -w "fichier" ]; then # -w = modifiable
if [ -x "fichier" ]; then # -x = executable
if [ -s "fichier" ]; then # -s = non vide

# Tests sur chaines
if [ "$var" = "valeur" ]; then # Egalite chaine
if [ "$var" != "valeur" ]; then # Difference
if [ -z "$var" ]; then # Chaine vide
if [ -n "$var" ]; then # Chaine non vide

# Tests numeriques
if [ "$a" -eq "$b" ]; then # Egal (equal)
if [ "$a" -ne "$b" ]; then # Different (not equal)
if [ "$a" -gt "$b" ]; then # Superieur (greater than)
if [ "$a" -lt "$b" ]; then # Inferieur (less than)
if [ "$a" -ge "$b" ]; then # Superieur ou egal
if [ "$a" -le "$b" ]; then # Inferieur ou egal
```

Boucle for

```
#!/bin/bash
# Boucle for sur une liste
for i in 1 2 3 4 5; do
echo "Valeur : $i"
done

# Boucle for sur une plage
for i in $(seq 1 10); do
echo $i
done

# Boucle for sur le contenu d'une variable (exemple du cours)
x=$(ls $2) # Liste le contenu du repertoire $2
for i in $x; do
if test -f $2/$i; then
cat $1 > $2/$i # Remplace le contenu par celui de fich1
((nbf=nbf+1))
else
nbd=$(expr $nbd + 1)
fi
done

# Boucle for style C
for ((i=0; i<10; i++)); do
echo $i
done
```

Boucle while et until

```
#!/bin/bash
# while : boucle tant que la condition est vraie
compteur=0
while [ $compteur -lt 10 ]; do
echo "Compteur : $compteur"
((compteur++))
done

# Lecture ligne par ligne d'un fichier
while read ligne; do
echo "Ligne : $ligne"
done < fichier.txt

# until : boucle jusqu'a ce que la condition soit vraie
until [ $compteur -ge 10 ]; do
((compteur++))
done
```

■ case/esac

```
#!/bin/bash
case "$variable" in
"valeur1")
echo "Cas 1"
;;
"valeur2"|"valeur3")
echo "Cas 2 ou 3"
;;
*)
echo "Cas par defaut"
;;
esac
```

◆ 6.3 Script complet - Exemple du cours (remplace.sh)

Script qui remplace le contenu de chaque fichier d'un repertoire par celui de fich1 et compte les elements :

```
#!/bin/bash
# Usage : ./remplace.sh fich1 rep
# $1 = fichier source, $2 = repertoire cible

nbf=0 # compteur de fichiers
nbd=0 # compteur de repertoires

x=$(ls $2) # liste le contenu du repertoire $2

for i in $x; do
if test -f $2/$i; then
cat $1 > $2/$i # remplace le contenu
((nbf=nbf+1))
else
nbd=$(expr $nbd + 1) # incremente compteur repertoires
fi
done
```

```
echo "$nbf fichiers et $nbd repertoires dans $2"
```

Pour rendre un script executable et l'executer :

```
chmod u+x remplace.sh # Rendre executable
./remplace.sh fich1 rep # Executer

# Ou directement avec bash :
bash remplace.sh fich1 rep
sh remplace.sh fich1 rep
```

[+] A RETENIR

Le shebang #!/bin/bash est la premiere ligne obligatoire d'un script.

Pas d'espace autour de = pour les variables : var="valeur" (correct), var = "valeur" (erreur).

$1, $2... : arguments du script. $# : nombre d'arguments. $? : code retour.

test -f fichier = fichier ordinaire. test -d = repertoire. -e = existe.

Pour executer : chmod u+x script.sh puis ./script.sh, ou bash script.sh directement.

[!] ATTENTION AUX PIEGES

Les espaces dans [condition] sont obligatoires : [-f "fich"] et non [-f "fich"].

Toujours mettre les variables entre guillemets : "$var" et non $var.

La substitution de commande : x=$(cmd) est preferee a x=`cmd` (backticks moins lisibles).

■ 8. EXERCICES CORRIGES - TD DE REVISION

◆ Exercice 1 - QCM sur les permissions (corrige)

Contexte :

```
Utilisateurs et groupes :
consultant1 : groupes consultant1, database1
operator1 : groupes operator1, database1
contractor1 : groupes contractor1, contractor3
operator2 : groupes operator2, contractor3

Fichiers dans le repertoire courant :
drwxrwxr-x. operator1 database1 . (repertoire courant)
-rw-rw-r--. consultant1 consultant1 lfile1
-rw-r--rw-. consultant1 database1 lfile2
-rw-rw-r--. operator1 database1 rfile1
-rw-r-----. operator1 database1 rfile2
drwxrwsr-t. operator1 database1 rep (SGID+Sticky)
-rwSr--rw-. consultant1 database1 file5 (SUID actif, x absent)
```

Q1	Fichier dont le proprietaire est operator1 et lisible par tous ?
[OK] Reponse :	**c. rfile1**
[?] Explication :	rfile1 : -rw-rw-r-- -> proprietaire=operator1, autres=r-- (lisible). rfile2 : -rw-r----- -> autres=--- (non lisible)

Q2	Fichier modifiable par contractor1 ?
[OK] Reponse :	**b. lfile2**
[?] Explication :	contractor1 est dans {contractor1, contractor3}. lfile2 : -rw-r--rw- -> others=rw- -> contractor1 est "autre" -> peut ecrire

Q3	Fichier non lisible par operator2 ?
[OK] Reponse :	**d. rfile2**
[?] Explication :	operator2 est dans {operator2, contractor3}. rfile2 : -rw-r----- -> group=database1, operator2 n'est pas dans database1 -> autres=--- -> non lisible

Q4	Fichiers supprimables par operator1 dans le repertoire . ?
[OK] Reponse :	**c. rfile1 et rfile2**
[?] Explication :	La suppression depend des droits sur le REPERTOIRE PARENT. operator1 est dans database1 -> droits rwx sur . -> peut supprimer.

Q5	consultant1 peut-il executer file5 ?

[OK] Reponse :	**b. Faux**
[?] Explication :	file5 : -rwSr--rw- -> S majuscule = SUID actif MAIS x absent pour le proprietaire. consultant1 est le proprietaire -> droits = rwS = rw- (S ne donne pas x). Non executable.

Q6	Groupe des fichiers crees par operator1 dans rep ?
[OK] Reponse :	**a. database1**
[?] Explication :	rep : drwxrwsr-t -> le s (SGID) est actif. Avec SGID, les nouveaux fichiers heritent du groupe du repertoire = database1.

◆ Exercice 2 - Gestion complete utilisateurs/groupes/permissions (corrige)

Question 1 : Creez deux groupes techdocs et consultants et deux utilisateurs user1 et user2.

```
groupadd techdocs
groupadd consultants
useradd user1
useradd user2
```

Question 2 : Creez un utilisateur engineer avec techdocs comme groupe secondaire, /bin/sh comme shell et consultants comme groupe primaire.

```
useradd engineer -G techdocs -s /bin/sh -g consultants
```

Question 3 : Donnez a l'utilisateur engineer le privilege d'utiliser sudo.

```
usermod -aG wheel engineer
# Sur certains systemes : usermod -aG sudo engineer
```

Question 4 : Creez en tant que engineer un repertoire /home/techdocs. Changez son groupe proprietaire a techdocs.

```
su - engineer
mkdir -p /home/techdocs
chgrp techdocs /home/techdocs
```

Question 5 : Modifier les permissions sur /home/techdocs : engineer peut lire, ecrire et acceder.

```
chmod u=rwx /home/techdocs
```

Question 6 : Interdire la suppression des fichiers dans /home/techdocs par tout utilisateur (sauf engineer et root).

```
chmod o+t /home/techdocs
# Active le sticky bit
```

Question 7 : Les fichiers crees dans /home/techdocs auront techdocs comme groupe proprietaire.

```
chmod g+s /home/techdocs
# Active le SGID : heritage du groupe
```

Question 8 : Definissez l'expiration du compte de engineer au 12-12-2024.

```
chage -E 2024-12-12 engineer
```

Question 9 : Forcez user1 a changer son mot de passe lors de la premiere connexion.

```
chage -d 0 user1
```

Question 10 : Configurez les permissions de user2 sur /home/techdocs : rwx (ACL).

```
setfacl -m u:user2:rwx /home/techdocs
```

Question 11 : Copiez dans /home/techdocs tous les fichiers de /usr dont le nom finit par une minuscule.

```
find /usr -name "*[a-z]" -exec cp {} /home/techdocs \;
```

Question 12 : Supprimez tous les repertoires vides dans /home.

```
find /home -type d -empty -exec rmdir {} \;
```

Question 13 : Dans /usr, trouvez les fichiers > 1Mo avec droits 755. Sauvegardez dans names.

```
find /usr -size +1M -a -perm 755 > names
```

Question 14 : Dans names, remplacez les minuscules par des majuscules. Affichez la 8eme ligne.

```
tr "[a-z]" "[A-Z]" < names > names2
mv names2 names
cat names | head -n8 | tail -1
# Ou avec sed (en place) :
sed -i 's/.*/\U&/g' names
cat names | head -n8 | tail -1
```

Question 15 : Supprimez les redondances de names. Sauvegardez dans new.names.

```
cat names | sort | uniq > new.names 2>/dev/null
```

◆ Exercice 3 - Filtres et sed (corrige)

Fichier couleurs :

```
# Couleurs
Rouge
#
Vert
# ?
Bleu
# Jaune
Beige
Noir /
Blanc Bleu Rouge Bleu
```

Q1 : Supprimez les lignes contenant #.

```
sed '/^#/d' couleurs
# /^#/ = motif : ligne commencant par #, d = delete
```

Q2 : Affichez les lignes finissant par e.

```
grep 'e$' couleurs
# Ou : sed -n '/e$/p' couleurs
```

Q3 : Remplacez # par un espace.

```
sed 's/#/ /g' couleurs
# s/ancien/nouveau/g remplace toutes les occurrences
```

Q4 : Remplacez ? par *.

```
sed 's/?/*/g' couleurs
```

Q5 : Extrait les GIDs de /etc/passwd, sans doublons.

```
cut -d: -f4 /etc/passwd | sort -n | uniq
```

◆ Exercice 4 - Commandes find (corrige)

Q1 : Fichiers terminant par .c, erreurs vers /dev/null.

```
find / -name "*.c" 2>/dev/null
```

Q2 : Fichiers dans /usr de taille > 1Mo avec droits 755.

```
find /usr -size +1M -a -perm 755
```

Q3 : Compter les fichiers appartenant a student avec droits 666.

```
find / -user student -a -perm 666 | wc -l
```

Q4 : Dans /home, fichiers commencant par minuscule ou P majuscule.

```
find /home -name "[a-z,P]*"
```

■ 9. QUESTIONS D'EXAMEN FREQUEMMENT POSEES

◆ Categorie : Permissions

[?] **Quelle est la notation octale de -rwxr--r-x ?**

[OK] 745 (r=4, w=2, x=1 -> rwx=7, r--=4, r-x=5)

[?] **Comment lire une ligne ls -l ?**

[OK] Type(1) + droits_user(3) + droits_groupe(3) + droits_autres(3) + [.] + proprietaire + groupe + nom

[?] **Que signifie drwxrwsr-t ?**

[OK] d=repertoire, rwx=user, rws=groupe(SGID actif), r-t=autres(sticky). Protection suppression + heritage groupe.

[?] **Difference entre chmod u+s et chmod g+s sur un repertoire ?**

[OK] u+s (SUID) : peu d'effet sur repertoire. g+s (SGID) : les nouveaux fichiers heritent du groupe du repertoire.

[?] **Comment supprimer un fichier dans un repertoire avec sticky bit ?**

[OK] Seul le proprietaire du fichier, le proprietaire du repertoire, ou root peut le supprimer.

◆ Categorie : Utilisateurs

[?] **Structure de /etc/passwd ?**

[OK] login:x:uid:gid:commentaire:home_directory:shell (7 champs separes par :)

[?] **Structure de /etc/group ?**

[OK] nom_groupe:x:gid:membres (membres separes par virgules)

[?] **Difference -g et -G dans useradd ?**

[OK] -g = groupe PRIMAIRE (un seul), -G = groupes SECONDAIRES (liste separee par virgules)

[?] **Que fait chage -d 0 user ?**

[OK] Force user a changer son mot de passe a la prochaine connexion (date dernier changement = 0)

[?] **Que fait usermod -aG wheel engineer ?**

[OK] Ajoute engineer au groupe wheel (sans -a, ca remplacerait tous ses groupes secondaires !)

◆ Categorie : Redirections & Filtres

[?] **Difference entre > et >> ?**

[OK] > redirige en ecrasant. >> redirige en ajoutant a la fin du fichier.

[?] **Comment rediriger stdout ET stderr vers un fichier ?**

[OK] cmd > fichier 2>&1 (d'abord stdout, puis stderr vers la ou est stdout)

[?] **Comment supprimer tous les doublons d'un fichier ?**

[OK] sort fichier | uniq > nouveau_fichier (sort prealable obligatoire pour uniq)

[?] **Comment afficher la Nieme ligne d'un fichier ?**

[OK] cat fichier | head -nN | tail -1

[?] **Comment remplacer les minuscules par des majuscules ?**

[OK] tr "[a-z]" "[A-Z]" < fichier ou sed 's/.*/\U&/g' fichier

◆ Categorie : Shell

[?] **Comment rendre un script executable et l'executer ?**

[OK] chmod u+x script.sh puis ./script.sh ou directement bash script.sh

[?] **Difference entre test -f et test -d ?**

[OK] test -f = fichier ordinaire. test -d = repertoire. test -e = existe (fichier ou repertoire).

[?] **Comment recuperer le code de retour de la derniere commande ?**

[OK] $? (0 = succes, autre valeur = erreur)

[?] **Difference entre $@ et $* ?**

[OK] $@ : chaque argument est une chaine separee. $* : tous les arguments forment une seule chaine.

◆ Categorie : Processus

[?] **Difference entre kill -9 et kill -15 ?**

[OK] kill -15 (SIGTERM) : arret propre, le processus peut l'intercepter. kill -9 (SIGKILL) : arret force immediat, ne peut etre ignore.

[?] **Que fait Ctrl+Z vs Ctrl+C ?**

[OK] Ctrl+Z : suspend le processus (SIGSTOP). Ctrl+C : interrompt le processus (SIGINT).

[?] **A quoi sert nohup ?**

[OK] Permet a une commande de continuer meme apres deconnexion (immunise contre SIGHUP). Usage : nohup cmd &

■ 10. FICHE DE REVISION RAPIDE

◆ Permissions - Tableau de conversion rapide

Octal	Binaire	Symbolique	Octal	Binaire	Symbolique
0	000	---	4	100	r--
1	001	--x	5	101	r-x
2	010	-w-	6	110	rw-
3	011	-wx	7	111	rwx

◆ Permissions speciales - Octets de prefixe

Bit	Valeur octale	Exemple chmod	Resultat ls -l
Sticky	1	chmod 1755 /rep ou chmod o+t /rep	drwxr-xr-t
SGID	2	chmod 2755 /rep ou chmod g+s /rep	drwxr-sr-x
SUID	4	chmod 4755 prog ou chmod u+s prog	-rwsr-xr-x
SGID+Sticky	3	chmod 3775 /rep	drwxrwsr-t

◆ Commandes incontournables - Memo examen

[F] FICHIERS

- ls -l -> liste detaillee
- cp -r -> copie recursive
- rm -rf -> suppression forcee
- find -exec -> cherche+execute
- ln -s -> lien symbolique

[R] FILTRES

- grep "motif" fich -> cherche
- sed 's/a/b/g' -> remplace
- cut -d: -f1 -> colonne
- sort | uniq -> deduplique
- tr "a-z" "A-Z" -> majuscules

[U] UTILISATEURS

- useradd -g -G -s -d
- usermod -aG -> ajout groupe
- userdel -r -> + home
- chage -E/-M/-m/-W/-d 0
- passwd user -> mot de passe

[P] PERMISSIONS

- chmod u+x / chmod 755
- chown user:grp fichier
- chgrp -R grp /rep
- setfacl -m u:user:rwx
- getfacl -> lire ACL

[Pr] PROCESSUS

- ps aux -> tous processus
- kill -9 PID -> force arret
- jobs/fg/bg -> gestion jobs
- nohup cmd & -> resistant deco
- Ctrl+Z suspend, Ctrl+C tue

[S] SHELL

- #!/bin/bash -> shebang
- $1,$2,$#,$? -> parametres
- if [-f/-d/-e] -> tests
- for i in $(ls) -> boucle
- chmod u+x -> rendre executable

■ 11. GLOSSAIRE LINUX

ACL (Access Control List)	Liste de controle d'acces permettant d'attribuer des droits a des utilisateurs/groupes supplementaires en dehors du modele UGO classique. Geree avec setfacl/getfacl.
Arriere-plan (Background)	Mode d'execution d'un processus qui n'occupe pas le terminal. Lance avec & ou deplace avec bg.
Bash	Bourne Again SHell - shell par defaut sur la plupart des distributions Linux. Interpreteur de commandes et langage de script.
Chemin absolu	Chemin commencant par / (racine) : /home/user/fichier. Independant du repertoire courant.
Chemin relatif	Chemin relatif au repertoire courant : ../fichier ou ./script.sh.
chage	Commande de gestion de la politique d'expiration des mots de passe et des comptes.
chmod	CHange MODe - commande pour modifier les permissions d'acces d'un fichier ou repertoire.
chown	CHange OWNer - commande pour changer le proprietaire d'un fichier ou repertoire.
Demon (Daemon)	Processus en arriere-plan sans interaction utilisateur directe (serveur web, cron, sshd...).
FHS	Filesystem Hierarchy Standard - norme definissant l'arborescence des repertoires sous Linux/Unix.
GID	Group IDentifier - numero unique identifiant un groupe dans le systeme.
grep	Global Regular Expression Print - commande de recherche de motifs dans des fichiers texte.
Groupe primaire	Groupe associe a un utilisateur lors de la connexion. Defini dans /etc/passwd (champ gid).
Groupe secondaire	Groupes supplementaires auxquels appartient un utilisateur. Definis dans /etc/group.
Hard link (lien physique)	Plusieurs noms pointant vers le meme inode. Limite au meme systeme de fichiers. Cree avec ln.
Inode	Structure de donnees contenant les metadonnees d'un fichier (permissions, proprietaire, dates...). Le nom du fichier n'est PAS dans l'inode.
Noyau (Kernel)	Coeur du systeme d'exploitation Linux. Gere les ressources materielles, la memoire, les processus.
PID	Process IDentifier - numero unique identifiant un processus en cours d'execution.

Pipe (|)	Mecanisme de communication entre processus : la sortie d'un processus devient l'entree du suivant.
PPID	Parent Process IDentifier - PID du processus pere.
Processus zombie	Processus termine mais dont le pere n'a pas encore recupere le code de retour. Visible avec ps comme [Z].
root	Superutilisateur du systeme Linux (UID=0). Dispose de tous les droits. Son repertoire home est /root.
sed	Stream EDitor - editeur de flux pour transformer du texte. Traite les fichiers ligne par ligne.
setuid (SUID)	Bit special qui permet a un programme de s'executer avec les droits de son proprietaire.
setgid (SGID)	Sur un fichier : s'execute avec les droits du groupe. Sur un repertoire : les nouveaux fichiers heritent du groupe du repertoire.
Shell	Interpreteur de commandes faisant l'interface entre l'utilisateur et le noyau. Bash est le plus courant.
Signal	Mecanisme de communication entre processus (ex: SIGTERM=15, SIGKILL=9, SIGINT=2). Envoye avec kill.
Sticky bit	Sur un repertoire : empeche les utilisateurs de supprimer les fichiers des autres. Utilise sur /tmp.
stdin / stdout / stderr	Flux standard : entree (0), sortie (1), erreur (2). Redirigibles avec <, >, 2>.
Symbolic link (lien symbolique)	Raccourci pointant vers un fichier/repertoire. Peut traverser les systemes de fichiers. Cree avec ln -s.
UID	User IDentifier - numero unique identifiant un utilisateur. root=0, utilisateurs normaux >= 1000.
umask	Masque de permissions par defaut a la creation de fichiers. umask 022 -> nouveaux fichiers en 644.

FIN DU GUIDE DE REVISION LINUX

Ce document compile les cours Chapitres 1 a 6, les notes de revision, les TD corriges et les anciens examens. Bonne revision !